

Container mit Schwerpunkt Kubernetes

Harbor als Container Artifactory



Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnarability-Scanner
- Image Signing
- Policy Enforcement

Agenda

Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnerability-Scanner
- Image Signing
- Policy Enforcement

Kubernetes als Standardplattform – und seine blinden Flecken

Kubernetes hat sich in den letzten Jahren als De-facto-Standard für den Betrieb containerisierter Anwendungen etabliert. Unternehmen aller Größen nutzen Kubernetes, um Anwendungen skalierbar, hochverfügbar und automatisiert zu betreiben.

In vielen Einführungen liegt der Fokus dabei auf technischen Kernkonzepten wie Pods, Deployments, Services oder Autoscaling. Diese Aspekte sind zweifellos wichtig – sie betreffen jedoch primär die **Laufzeit** von Anwendungen.

Was dabei häufig zu kurz kommt, ist eine grundlegende Frage:

- **Wie vertrauenswürdig sind die Anwendungen, die Kubernetes ausführt?**
- Kubernetes selbst trifft dazu zunächst **keine Aussage**.
- Die Plattform geht implizit davon aus, dass die bereitgestellten Container Images korrekt, sicher und vertrauenswürdig sind.
- Diese Annahme ist in modernen Software-Lieferketten jedoch zunehmend kritisch.

Container Images als Herzstück der Software-Lieferkette

Ein Container Image ist weit mehr als nur der eigentliche Anwendungscode.

Es ist ein **vollständiges Softwareartefakt**, das typischerweise folgende Bestandteile enthält:

- ein Base Image (z. B. Linux-Distribution oder Runtime)
- Betriebssystem- und Systembibliotheken
- Applikationsabhängigkeiten (Frameworks, Packages)
- den eigenen Anwendungscode
- Konfigurations- und Startlogik

Damit bildet das Container Image den **exakten technischen Zustand** einer Anwendung zu einem bestimmten Zeitpunkt ab.

In Kubernetes gilt dabei ein zentraler Grundsatz:

Jeder Pod, jeder Container, jede Replik basiert direkt auf einem Container Image.

Ein fehlerhaftes oder kompromittiertes Image wird nicht einmal, sondern potenziell **hundertfach oder tausendfach** im Cluster ausgeführt.

Kubernetes prüft Images nicht – und das ist Absicht

Ein häufiges Missverständnis ist die Annahme, Kubernetes würde Images automatisch auf Sicherheit oder Qualität prüfen.

Das ist **nicht der Fall** – und bewusst so entworfen.

Kubernetes:

- überprüft keine Schwachstellen
- prüft keine Herkunft eines Images
- validiert keine Integrität
- erzwingt keine Sicherheitsrichtlinien

Der Grund dafür ist die klare Trennung von Verantwortlichkeiten:

- Kubernetes ist eine **Orchestrierungsplattform**
- Sicherheits- und Governance-Entscheidungen müssen **außerhalb** getroffen werden

Damit entsteht jedoch eine zentrale Frage:

Wo und wie stellen wir sicher, dass nur vertrauenswürdige Images im Cluster landen?

Die Container Registry als zentraler Kontrollpunkt

In jeder Kubernetes-Umgebung existiert ein unvermeidbarer Übergabepunkt zwischen Entwicklung und Betrieb:

Quellcode → CI/CD → Container Image → Registry → Kubernetes

Die **Container Registry** ist der Ort, an dem:

- Images erstmals zentral verfügbar werden.
- Images versioniert und verteilt werden.
- Images von Kubernetes konsumiert werden.

Sie ist damit ein idealer Punkt für:

- Sicherheitsprüfungen
- Zugriffskontrolle
- Richtlinien und Governance
- Nachvollziehbarkeit und Auditierbarkeit

Governance beschreibt die Gesamtheit der Regeln, Verantwortlichkeiten und technischen Durchsetzungsmechanismen, die sicherstellen, dass Container Images kontrolliert, nachvollziehbar und regelkonform erstellt, freigegeben und betrieben werden.

Governance beantwortet dabei nicht die Frage „*Wie deployen wir?*“, sondern:

- Wer darf was?
- Unter welchen Bedingungen?
- Nach welchen Regeln?
- Wie wird der Prozess technisch sichergestellt und nachvollziehbar gemacht?

In vielen Umgebungen wird dieser Punkt jedoch nur als **reiner Speicher** betrachtet – mit erheblichen Risiken.

Grenzen klassischer und öffentlicher Registries

Öffentliche Registries wie Docker Hub oder einfache private Registries erfüllen primär eine Aufgabe:

Images speichern und bereitstellen

Für produktive Kubernetes-Plattformen fehlen dabei oft entscheidende Funktionen:

- Zugriffskontrolle
- integrierten Sicherheitsanalysen
- Aussage über Herkunft und Integrität
- Möglichkeit, unsichere Images aktiv zu blockieren
- Audit- und Compliance-Funktionen

Damit entsteht eine Sicherheitslücke zwischen CI/CD und Kubernetes, die nicht durch Kubernetes selbst geschlossen werden kann.

Harbor: Mehr als eine Container Registry

Harbor wurde entwickelt, um diese Lücke zu füllen.

Harbor ist eine **Enterprise-fähige, Open-Source Container Registry (CNCF-Projekt)**.

Als Ergänzung zur klassischen Registry versteht sich Harbor als **Security- und Governance-Komponente** innerhalb der Container-Lieferkette.

Harbor erweitert die Registry um folgende Funktionalitäten:

- strukturierte Organisation von Images (Projects)
- rollenbasierte Zugriffskontrolle
- automatische Vulnerability Scans
- kryptografische Image-Signaturen
- Richtlinien zur Freigabe oder Blockierung von Images
- Audit- und Nachvollziehbarkeit

Harbor ist kein passiver Speicher, sondern ein aktiver Gatekeeper für Container Images.

Harbor im Zusammenspiel mit Kubernetes

Für das Verständnis ist eine klare Rollenverteilung entscheidend:

- **Harbor entscheidet**, welche Images als vertrauenswürdig gelten.
- **Kubernetes setzt durch**, welche Images ausgeführt werden dürfen.

Harbor arbeitet dabei **vor dem Deployment**

- beim Push eines Images (in die Registry),
- bei der Bewertung von Schwachstellen,
- bei der Prüfung von Signaturen,
- bei der Durchsetzung von Policies.

Kubernetes greift diese Entscheidungen beim Deployment auf, z. B. über:

- Admission Controller (→ ist der Mechanismus)
- Policy Engines (OPA, Kyverno) (→ definiert die Regeln für den Mechanismus)
- Image-Pull-Mechanismen

So entsteht eine kontrollierte, überprüfbare und automatisierte Sicherheitskette.

Harbor im Zusammenspiel mit Kubernetes

Was bedeutet „Artifactory“?

Der Begriff **Artifactory** leitet sich vom Wort „**Artefakt**“ ab.

In der Softwareentwicklung bezeichnet ein Artefakt ein **fertig erzeugtes Ergebnis** eines Build- oder Entwicklungsprozesses.

Typische Software-Artefakte sind zum Beispiel:

- kompilierte Programmdateien
- Bibliotheken (z. B. JAR-, DLL-, oder Pakete)
- Installationspakete (z. B. DEB, RPM)
- Container Images

Ein Artefakt ist dabei immer:

- eindeutig versioniert
- reproduzierbar erzeugt
- zur Weiterverwendung bestimmt

Harbor im Zusammenspiel mit Kubernetes

Eine **Artifactory** ist ein zentrales System zur **Ablage, Verwaltung und kontrollierten Nutzung** solcher Artefakte.

Sie stellt sicher, dass Software nicht nur gespeichert, sondern **versioniert, nachvollziehbar und regelbasiert** bereitgestellt wird.

Mit der Einführung von Containern ist das **Container Image** zum wichtigsten Artefakt moderner Plattformen geworden:

Jedes Deployment in Kubernetes basiert unmittelbar auf einem Container Image.

Eine einfache Container Registry speichert Images.

Eine Artifactory hingegen betrachtet Images im Kontext ihres **Lebenszyklus** – von der Erstellung über Bewertung und Freigabe bis zur Nutzung in verschiedenen Umgebungen.

Harbor übernimmt genau diese Rolle für Container Images:

- zentrale Ablage
- strukturierte Organisation
- Sicherheits- und Vertrauensbewertungen
- Durchsetzung von Regeln und Verantwortlichkeiten

Damit ist Harbor nicht nur eine Registry, sondern die **Container-Artifactory im Kubernetes-Ökosystem**.

Harbor im Zusammenspiel mit Kubernetes

Vergleich: Harbor, Nexus Repository und Docker Hub

■ Grundsätzlicher Vergleich:

Aspekt	Docker Hub	Nexus Repository	Harbor
Primäre Aufgabe	Image-Verteilung	Artefaktverwaltung	Container-Artifactory
Zielgruppe	Public / Dev / OSS	CI/CD allgemein	Kubernetes-Plattformen
Artefakt-Fokus	Container Images	Viele Artefakttypen	Container Images
Einstiegshürde	Sehr niedrig	Mittel	Mittel
Kubernetes-Optimierung	Nein	Begrenzt	Ja

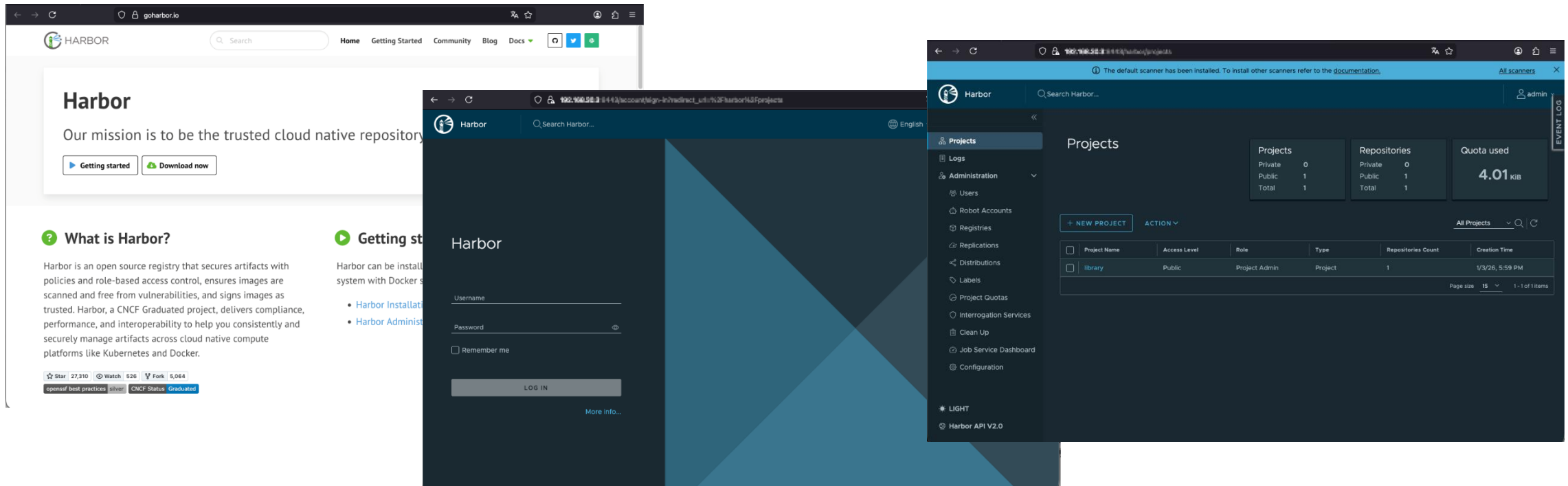
■ Security & Governance:

Funktion / Kriterium	Docker Hub	Nexus Repository	Harbor
Private Registry	Ja	Ja	Ja
Rollen & Zugriffskontrolle	Eingeschränkt	Ja	Fein granular (projektbasiert)
Vulnerability Scans	Sehr begrenzt	Optional / extern	Integriert
Image Signing / Trust	Nein	Begrenzt	Integriert
Policy Enforcement	Nein	Allgemein	Kubernetes-spezifisch
Audit & Nachvollziehbarkeit	Sehr begrenzt	Gut	Sehr gut

Harbor im Zusammenspiel mit Kubernetes

Harbor ist ein Open Source Projekt, das von der CNCF verwaltet wird. (<https://www.cncf.io>)

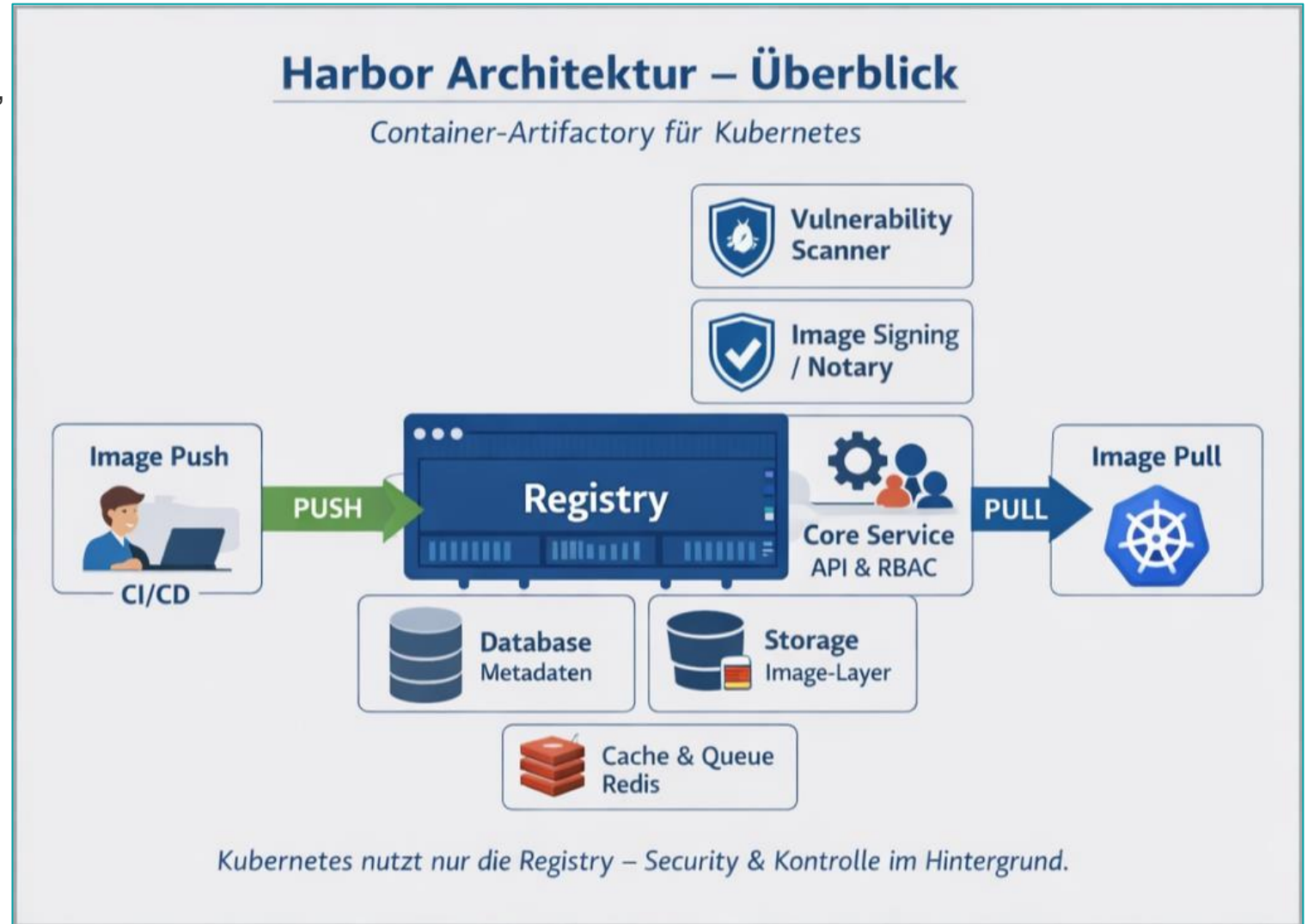
Harbor kann über die URL <https://goharbor.io/> runtergeladen werden und liegt aktuell (Feb./2026) in der Version 2.14.2 vor.



Harbor Architektur – kompakt (1)

Modularer Aufbau: Core Service, Registry, Scanner, Signing, Storage, DB, Redis

- **Trennung von Verantwortung:**
 - Core Service: Steuerung & RBAC
 - Registry: Push / Pull von Images
 - Scanner & Signing: Sicherheit & Vertrauensprüfung
 - DB & Storage: Metadaten & Image-Layer
 - **Kubernetes interagiert nur mit der Registry**



Harbor Architektur – kompakt (2)

Kernkomponenten

Komponente	Funktion
Core Service	API, Projekte, Benutzer, RBAC
Registry	Speicherung & Bereitstellung von Images
Scanner	Vulnerability-Analyse
Signing / Notary	Image-Vertrauen & Herkunft
Database	Metadaten & Scan-Ergebnisse
Storage	Persistente Image-Layer
Redis	Job-Queues & Performance

Überblick

Projekt

Container

Image

Registrierung

Netzwerk

Protokoll

Harbor installiert in Docker

Erstellen

Details

Aktion

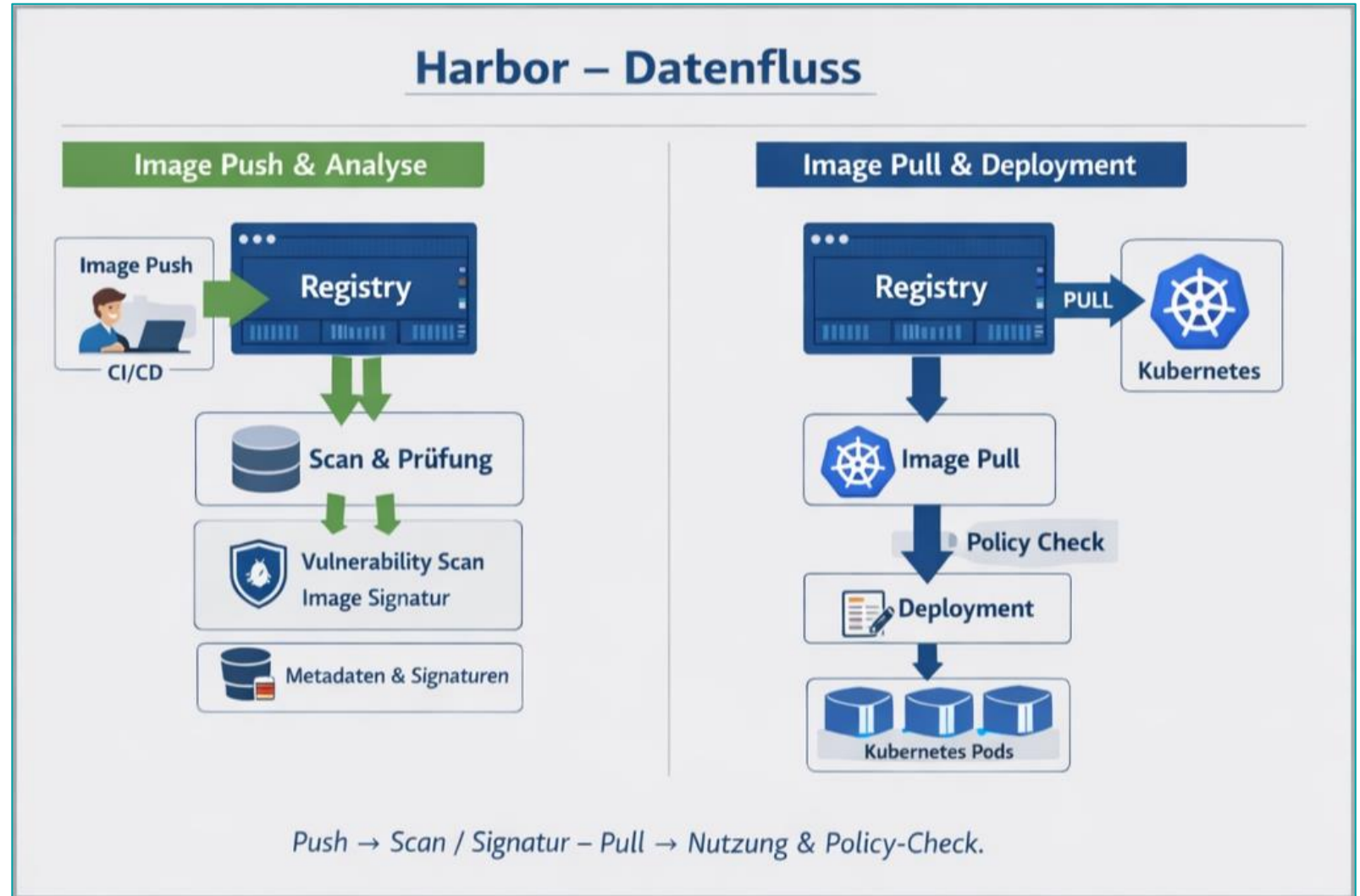
Status	Name	Image
●	nginx	goharbor/nginx-photon:v2.14.1
●	harbor-jobservice	goharbor/harbor-jobservice:v2.14.1
●	harbor-core	goharbor/harbor-core:v2.14.1
●	trivy-adapter	goharbor/trivy-adapter-photon:v2.14.1
●	harbor-db	goharbor/harbor-db:v2.14.1
●	registryctl	goharbor/harbor-registryctl:v2.14.1
●	harbor-portal	goharbor/harbor-portal:v2.14.1
●	redis	goharbor/redis-photon:v2.14.1
●	registry	goharbor/registry-photon:v2.14.1
●	harbor-log	goharbor/harbor-log:v2.14.1

Harbor installiert
in Docker

Harbor Architektur – kompakt (3)

Typischer Datenfluss

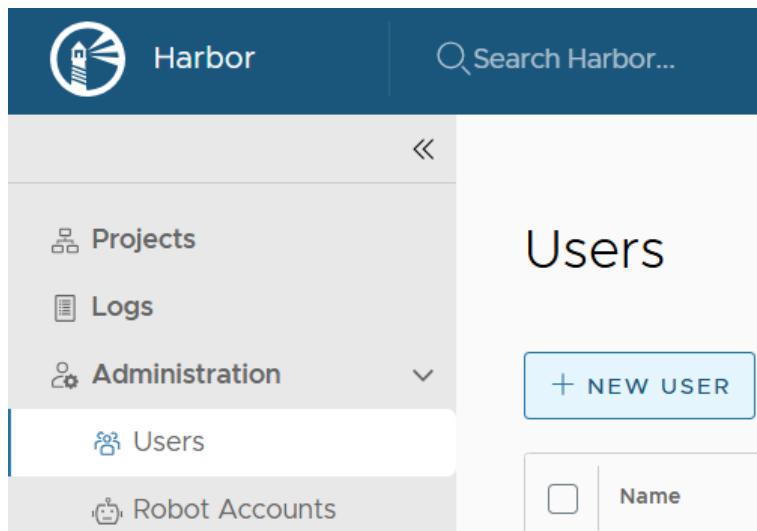
- **Push**
 - Image in Registry
 - Scan
 - Metadaten & Signaturen
- **Pull**
 - Kubernetes zieht Image
 - Kubernetes Policy: darf Image installiert werden?
- Kubernetes Policy ist optional.



User-Management

1. Grundidee

Aktionen in Harbor z. B push, pull werden durch einen User ausgeführt.



New User

Username *	project01Admin
Email *	project01admin@vm.local
First and last name *	Admin Project01
Password *	project01Admin
Confirm Password *	
Comments	Admin for project01

CANCELOK

Projekte als Sicherheits- und Organisationsgrenze

Policies und Governance

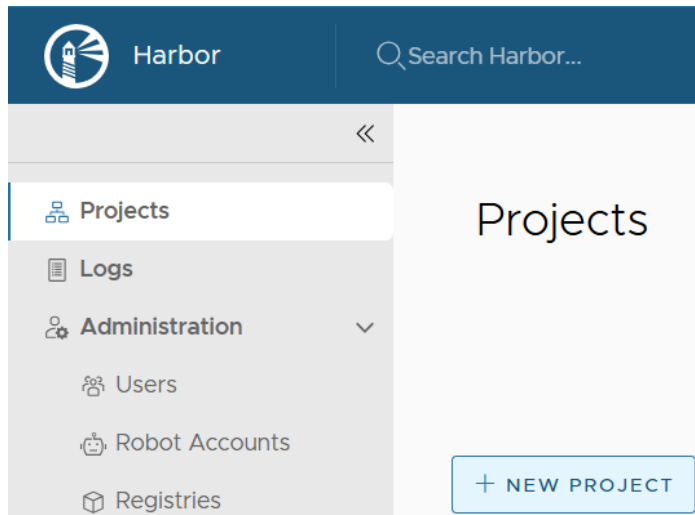
Projekte sind die **technische Grundlage**, um Regeln automatisch durchzusetzen:

- **Sicherheitsrichtlinien:** CRITICAL CVEs blockieren, nur geprüfte Base-Images zulassen.
- **Signaturprüfung:** nur signierte Images werden freigegeben.
- **Compliance & Audit:** Metadaten erfassen, wer was wann gepusht oder freigegeben hat.

Harbor Projekt einrichten

1. Grundidee

In Harbor ist ein **Projekt** die zentrale Einheit für **Organisation, Sicherheit und Governance**.



New Project

Project Name *

project01

Access Level ⓘ

☐ Public

Project quota limits ⓘ *

-1

GiB ▼

Proxy Cache ⓘ

☐

User zu Projekten hinzufügen

Projects

Logs

Administration

Users

Robot Accounts

Registries

Replications

Distributions

Labels

Projects

+ NEW PROJECT

ACTION

☐	Project Name	Access Level
☐	library	Public
☐	project01	Private

< Projects

project01

System

Summary

Repositories

Members

+ USER

+ GROUP

ACTION

☐	Name
☐	admin

Projekte als Sicherheits- und Organisationsgrenze

Rollen und Berechtigungen (RBAC)

Harbor erlaubt fein granulierte Rechte **pro Projekt**:

Rolle	Berechtigung / Nutzung
Guest	Nur Pull → z. B. Kubernetes
Developer	Push & Pull (Build → CI-Systeme)
Maintainer	Freigabe & Security-Review
Project Admin	Projektmanagement & Policies

Policies und Governance

Projekte sind die **technische Grundlage**, um Regeln automatisch durchzusetzen:

- **Sicherheitsrichtlinien:** CRITICAL CVEs blockieren, nur geprüfte Base-Images zulassen.
- **Signaturprüfung:** nur signierte Images werden freigegeben.
- **Compliance & Audit:** Metadaten erfassen, wer was wann gepusht oder freigegeben hat.

New Member

Add a user to be a member of this project with specified role

Name *

project01Admin

Role

☒ Project Admin

☐ Maintainer

☐ Developer

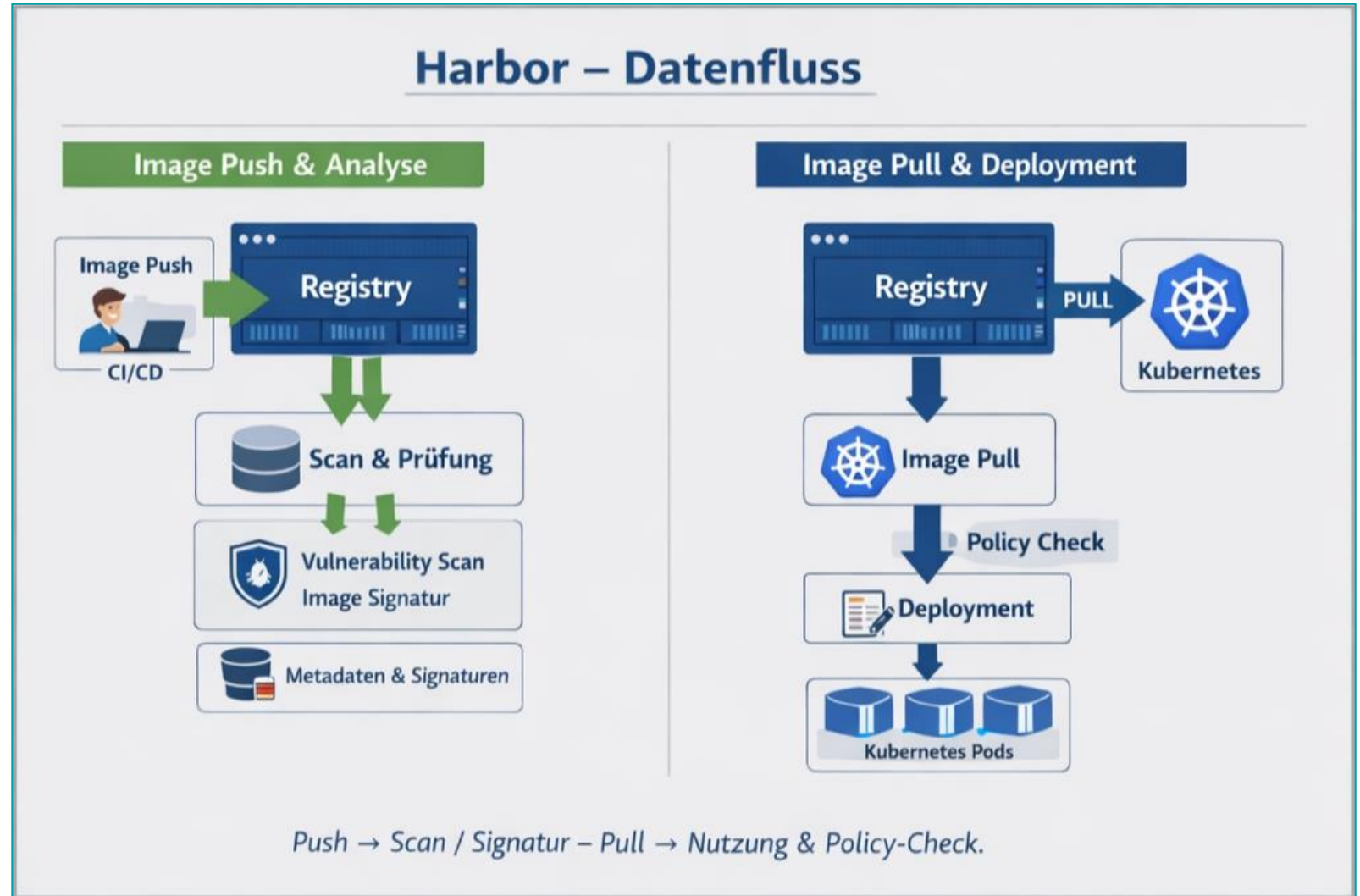
☐ Guest

☐ Limited Guest

Aufgabe: Harbor als zentrale Container-Artifactory nutzen (1)

Aufgaben:

- Docker-Secret erstellen
- Container-Image erstellen
- Image in Harbor pushen
- Image aus Harbor pullen



Aufgabe: Docker-Secret erstellen

Daten anpassen

- Host
- User
- Password

```
$ cat create-secret.sh
kubectl create secret docker-registry harbor-registry ... \
--docker-server=harbor.local ... \
--docker-username=project01Admin ... \
--docker-password=project01Admin
```

Aufgabe: Container-Image erstellen - mit buildah in Kubernetes (1)

- Ausgangspunkt: nginx
- individualisiert durch index.html
- Docker in ConfigMap
- index.html in ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: buildah-build-context
data:
  Dockerfile: |
    FROM docker.io/library/nginx:alpine
    COPY index.html /usr/share/nginx/html/index.html

  index.html: |
    <h1>Hello from Buildah + Harbor</h1>
```

Aufgabe: Container-Image erstellen - mit buildah in Kubernetes (2)

```
apiVersion: v1
kind: Pod
metadata:
  name: buildah-build
spec:
  restartPolicy: Never
  containers:
    - name: buildah
      image: quay.io/buildah/stable:latest
      securityContext:
        privileged: true
      command: ["/bin/sh", "-c"]
      args:
        - |
          cp /tmp/index.html /workspace/index.html
          buildah bud \
            --storage-driver=vfs \
            --isolation=chroot \
            -f /workspace/Dockerfile \
            -t harbor.local/project01/demo:1.0 \
            /workspace
```

```
    echo "Pushing image..."
    buildah push \
      --storage-driver=vfs \
      --authfile /tmp/auth/config.json \
      harbor.local/project01/demo:1.0
  volumeMounts:
    - name: config
      mountPath: /workspace/Dockerfile
      subPath: Dockerfile

    - name: config
      mountPath: /tmp/index.html
      subPath: index.html

    - name: harbor-secret
      mountPath: /tmp/auth
  volumes:
    - name: config
      configMap:
        name: buildah-build-context
    - name: harbor-secret
      secret:
        secretName: harbor-registry
        items:
          - key: .dockerconfigjson
            path: config.json
```

Aufgabe: Container-Image in Harbor überprüfen (1)

Harbor

Search Harbor...

Projects

Logs

< Projects

project01

Project Admin

Summary

Repositories

Members

Labels

X DELETE

☐ Name

project01/demo

Logs

Audit Logs

Audit Logs (Legacy)

Timestamp	Username	Resource	Resource Type	Operation	Operation Description
4/4/26, 6:42 AM	project01Admin	project01/demo:1.0	artifact	create	push artifact: project01/demo@sha256:f4d413032
4/3/26, 1:54 PM	admin	project01	project	create	create project: project01

demo

Info

Artifacts

SCAN VULNERABILITY

GENERATE SBOM

ACTIONS

Q | C

Artifacts	Tags	Signed	Size	Vulnerabilities	SBOM	Labels	Push Time	Pull Time
☐ sha256:f4d41303	☐ 1.0	☒	24.80MiB	Not Scanned	No SBOM		4/4/26, 6:42 AM	

Manage Columns

Page size 15 1 - 1 of 1 items

Aufgabe: Container-Image in Harbor überprüfen (2)

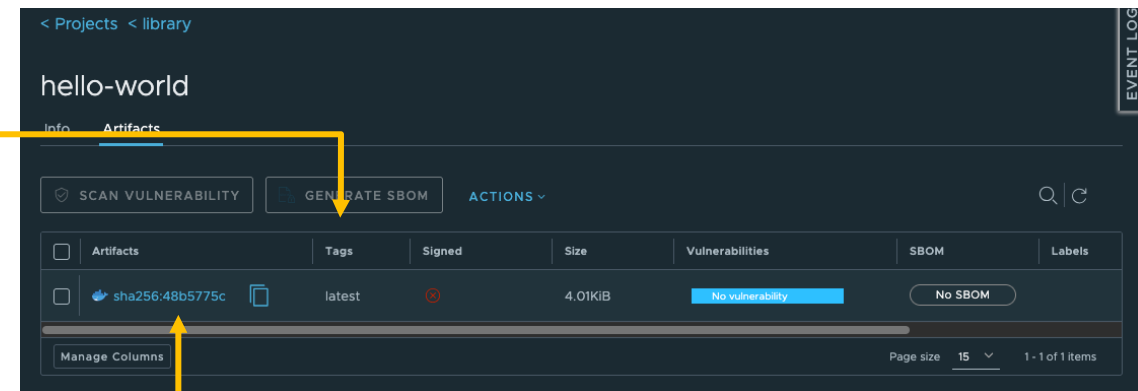
Befehl zum reproduzierbaren Deployment

Über das Kopiersymbol neben dem Artefakt-Namen ist der docker-Befehl verfügbar, zum Beispiel:

```
docker pull harbor.local/project01/demo@sha256:f4d41303a9d3a858beb99da883e6860ca56e797f3bc7d1712aad663ae8870712
```

Daten zum Artefakt

- **Repositories:** einzelne Anwendungen oder Services; Jedes Repository gehört einem Projekt
- **Tags / Versionen:** definieren deploybare Versionen; Versionierung erlaubt sichere Rollbacks und Updates
- **Digests:** unveränderliche Referenzen für reproduzierbare Deployments; Digests garantieren, dass ein Image nicht verändert wurde



Aufgabe: Container-Image pullen

- Pod nutzt Image aus harbor.
- Überprüfung des Images per curl.

```
apiVersion: v1
kind: Pod
metadata:
  name: demo-pod
spec:
  imagePullSecrets:
    - name: harbor-registry
  containers:
    - name: demo
      image: <Server>/<Projekt>/<Registry>:<Version>
      imagePullPolicy: Always
      ports:
        - containerPort: 80
```

```
$ kubectl exec -it demo-pod -- sh
/# curl http://localhost
<h1>Hello from Kaniko + Harbor</h1>
```

Manueller Import von Images aus externen Registries

a. Motivation und Praxisbezug

In nahezu allen Kubernetes-Umgebungen werden Container Images verwendet, die **nicht selbst entwickelt**, sondern aus **öffentlichen Registries** bezogen werden. Docker Hub ist dabei die mit Abstand häufigste Quelle für:

- Base Images (z. B. nginx, alpine, python)
- Infrastruktur-Komponenten
- Beispiel- und Demo-Anwendungen

Diese Images **direkt aus Docker Hub** in Kubernetes zu verwenden, ist technisch einfach – bringt jedoch **erhebliche Risiken** mit sich:

- Keine zentrale Sicherheitsprüfung
- Keine Kontrolle über Versionen und Updates
- Keine einheitlichen Zugriffsregeln
- Abhängigkeit von externen Verfügbarkeiten und Rate Limits

Ziel von Harbor: Externe Images unter **eigene Kontrolle** bringen.

Manueller Import von Images aus externen Registries

b. Was bedeutet „Image importieren“?

Beim Import eines Images wird dieses **einmal aus einer externen Registry bezogen** und anschließend **in Harbor gespeichert**.

Ab diesem Zeitpunkt gilt:

- Das Image liegt vollständig in der eigenen Registry
- Es unterliegt den gleichen Sicherheits- und Governance-Regeln wie eigene Images
- Kubernetes kann das Image ausschließlich aus Harbor beziehen

Anmerkung:

Harbor verändert das Image nicht – es kontrolliert dessen Nutzung.

Manueller Import von Images aus externen Registries

c. Beispiel-Szenario: NGINX aus Docker Hub

Ausgangslage:

- Öffentliches Image: nginx:1.29
- Quelle: Docker Hub
- Ziel: Harbor-Projekt base-images

Ziel:

- Zentrale Ablage von Basis-Images
- Einheitliche Nutzung durch alle Teams
- Automatische Sicherheitsanalyse

Manueller Import von Images aus externen

d. Manueller Import über Docker CLI

Schritt 1: Harbor vorbereiten (über das UI):

- Anlegen des Users
- Anlegen des Projekts (base-images)
- User dem Projekt hinzufügen und Zugriffsrechte setzen (bspw. als Maintainer)

Schritt 2: Image aus Docker Hub beziehen

```
> docker pull nginx:1.29
```

- Image wird aus Docker Hub geladen
- Keine Security-Prüfung durch Harbor
- Image ist lokal verfügbar

Schritt 3: Image für Harbor neu taggen

```
> docker tag nginx:1.29 \
    harbor.example.com/base-images/nginx:1.29
```

Durch das Tagging wird festgelegt:

- Ziel-Registry (Harbor)
- Ziel-Projekt (base-images)
- Version des Images

Verknüpft das lokale Image mit der Harbor-Projekt, aber noch kein Zugriff auf die Registry.

Manueller Import von Images aus externen

Schritt 4: Login an Harbor

```
> docker login harbor.example.com
```

- Ab jetzt soll mit Harbor interagiert werden
- Harbor prüft Benutzer, Passwort und Projektberechtigungen
- Docker speichert Zugangsdaten lokal

Schritt 5: Image in Harbor pushen

```
> docker push harbor.example.com/base-images/nginx:1.29
```

Jetzt passiert der eigentliche Import:

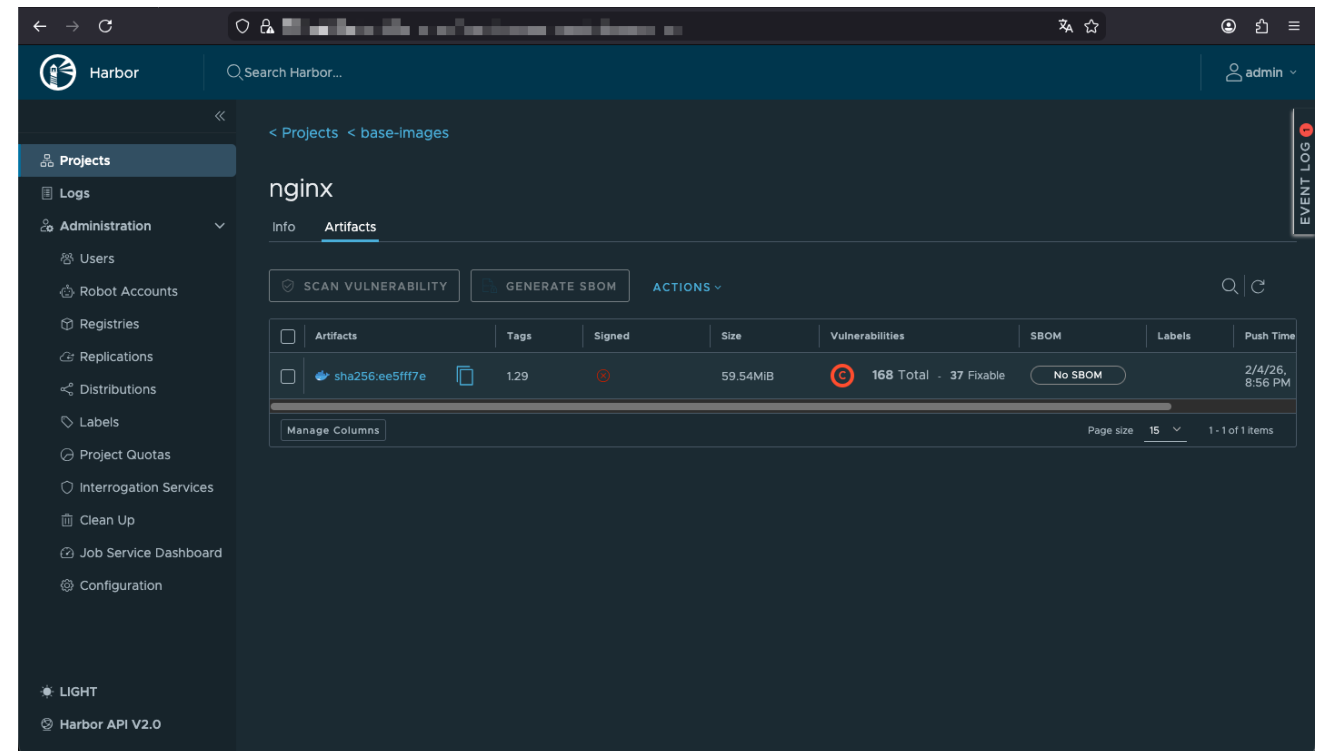
- Image wird in Harbor gespeichert
- Metadaten werden angelegt
- Vulnerability Scan wird ausgelöst
- Audit-Log wird geschrieben

Manueller Import von Images aus externen

e. Ergebnis in Harbor

Nach dem Push:

- Repository nginx im Projekt base-images
- Version 1.29
- Scan-Ergebnisse sichtbar *
- Einheitliche Nutzung durch Kubernetes möglich



* Nur wenn Trivy Scanner und Option 'scannen nach Push' für das Projekt aktiviert ist.

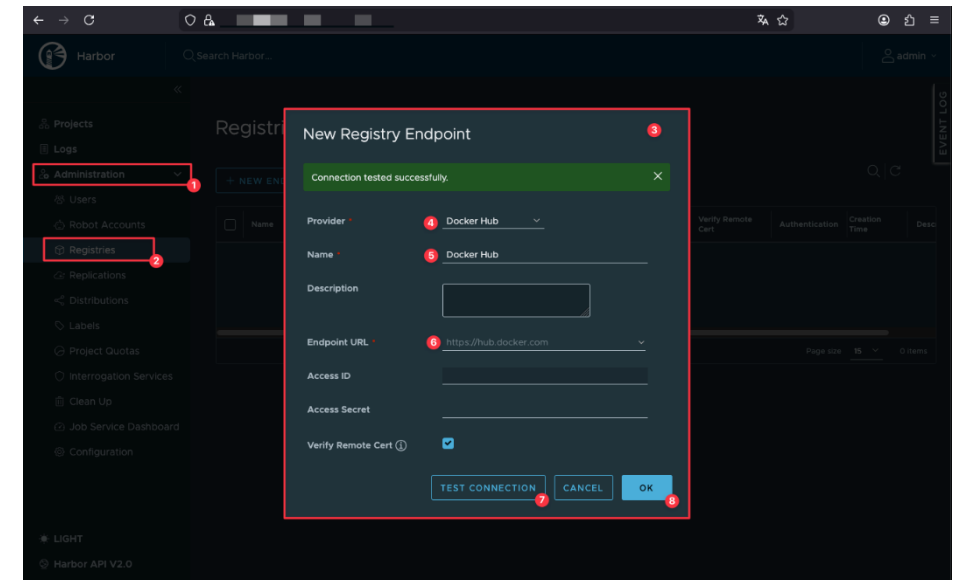
Automatisierter Import von Images aus externen Repositories

Automatisierter Import via Replikation

- Administrations-Rechte sind erforderlich.
- Projekt-Administratoren können keine Replikationen konfigurieren.

Schritt 1: Anlegen der Import-Quelle (über das UI):

- Unter Administration → Registries ‚New Endpoint‘ anklicken.
- Provider ‚Docker Hub‘ auswählen und neuen Namen vergeben.
- Unter ‚Endpoint Url‘ ‚https://hub.docker.com‘ eingeben.
- Access ID und Access Secret leer lassen da öffentlicher Zugriff.
- Mit ‚Test Connection‘ die Verbindung testen und anschließend mit ‚OK‘ speichern.

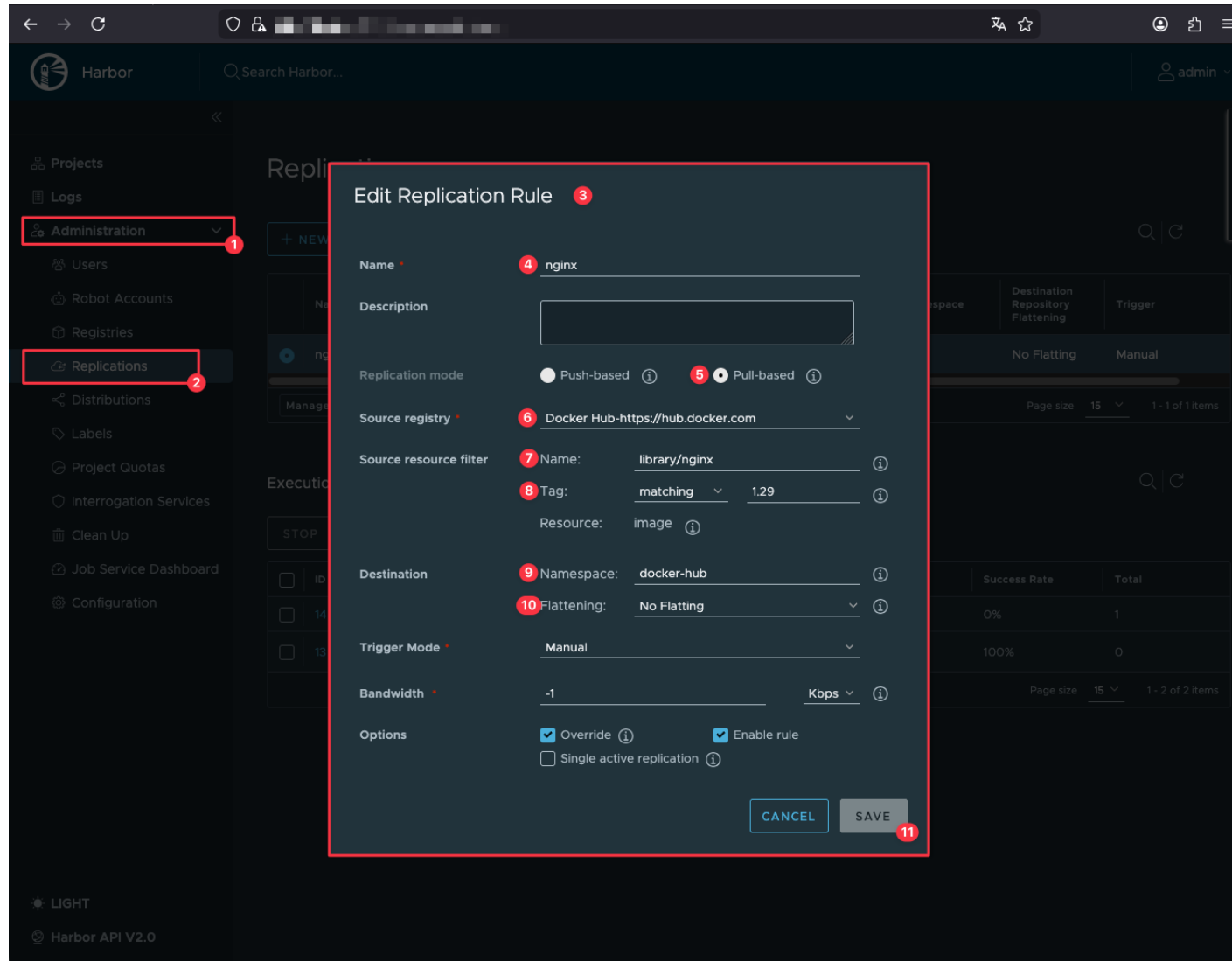


Automatisierter Import von Images aus externen

Schritt 2: Replikation anlegen (über das UI):

- Unter Administration → Replications ‚New Replication Rule‘ anklicken.
- Unter ‚Name‘ einen eigenen Replications-Namen vergeben (bspw. nginx).
- Unter ‚Replication Mode‘ Option ‚Pull-based‘ auswählen.
- Unter ‚Source Registry‘ den zuvor angelegten ‚Docker Hub‘ Eintrag wählen.
- Unter ‚Source Resource Filter – Name‘ die Bezeichnung des Images eingeben.
Achtung: hier muss ‚library/nginx‘ eingetragen werden.
- Unter ‚Source Resource Filter – Tag‘ den Eintrag ‚matching‘ auswählen und dahinter das Tag ‚1.29‘ eintragen.
- Unter ‚Destination‘ den Namen des Projektes (docker-hub) eintragen, in welches das Image importiert werden soll.
- Unter ‚Flattening‘ den Eintrag ‚No Flatting‘ auswählen.
- Zuletzt mit ‚Save‘ speichern.

Automatisierter Import von Images aus externen Registries



Harbor

Search Harbor...

admin

Administration

Users

Robot Accounts

Registries

Replications

Distributions

Labels

Project Quotas

Interrogation Services

Clean Up

Job Service Dashboard

Configuration

Execution

STOP

IP

14

13

LIGHT

Harbor API V2.0

Edit Replication Rule

Name: nginx

Description:

Replication mode: ☐ Push-based ☒ Pull-based

Source registry: Docker Hub-https://hub.docker.com

Source resource filter: Name: library/nginx Tag: matching 1.29 Resource: image

Destination: Namespace: docker-hub Flattening: No Flattening

Trigger Mode: Manual

Bandwidth: -1 Kbps

Options: ☒ Override ☒ Enable rule ☐ Single active replication

CANCEL SAVE

Automatisierter Import von Images aus externen Registries

Schritt 3: Replikation durchführen (über das UI):

- Unter Administration → Replications die gewünschte Replication auswählen.
- Anschließend ‚Replicate‘ klicken und in dem erscheinenden Fenster die Ausführung bestätigen.
- Im unteren Teil des Fensters kann der Fortschritt beobachtet und das Image im Projekt gefunden werden.

Replications

+ NEW REPLICATION RULE

REPLICATE

ACTIONS ▾

	Name	Status	Source registry	Replication Mode	Destination Registry:Namespace	Destination Repository Flattening	Trigger
☒	nginx	✓ Enabled	Docker Hub	pull-based	Local : docker-hub	No Flatting	Manual

Manage Columns

Page size 15

Executions

STOP

☐	ID	Status	Trigger	Start Time	Duration	Success Rate	Total
☐	3	Succeeded	manual	4/5/26, 7:38 AM	7m	100%	1

Page size 15

Automatisierter Import von Images aus externen Registries

Schritt 4: Verfügbarkeit überprüfen:

- Unter Projects das Projekt aussuchen, das zu dem Endpoint (Registries) gehört.
- Verfügbarkeit wie gewohnt prüfen.

[< Projects](#)



docker-hub

System Admin

[Summary](#)
[Repositories](#)
[Members](#)
[Labels](#)
[Scanner](#)
[P...](#)

×

DELETE

☐	Name	Artifacts
☐	docker-hub/library/nginx	1

[< Projects](#)
[< docker-hub](#)

library/nginx

[Info](#)
[Artifacts](#)



SCAN VULNERABILITY



GENERATE SBOM

ACTIONS

▼

☐	Artifacts	Tags	Signed	Size	Vulnerabilities
☐	 sha256:7150b3a3  	1.29		478.98MiB	Not Scanned

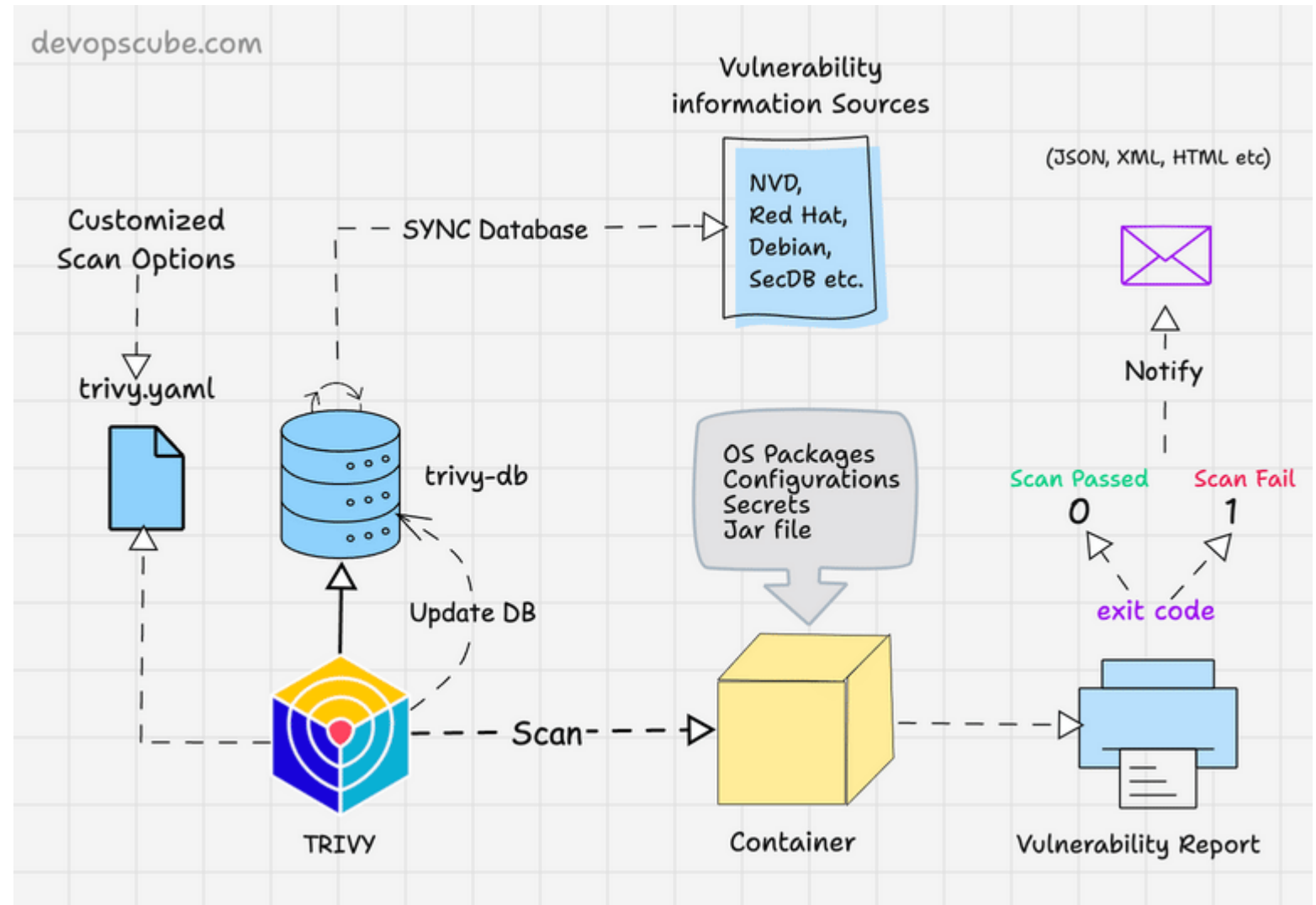
Agenda

Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnerability-Scanner
- Image Signing
- Policy Enforcement

Überblick Vulnerability Scan

- **Vulnerability Scan:** Container-Image wird auf Schwachstellen gescannt.
- Harbor verwendet **Trivy** als Vulnerability-Scanner.
- Trivy prüft
 - Base-Image
 - Installierte OS-Pakete
 - Verwendete Libraries
- Schwachstellen sind in CVE-Datenbanken zusammengestellt. CVE = *Common Vulnerabilities and Exposures*.
- Umgang mit dem Scan-Ergebnis wird anhand von **Policies** in Harbor konfiguriert.



Vulnerability Scanning - Sicherheit von Container Images bewerten

Bedeutung von Vulnerability Scanning im Kubernetes-Umfeld

- In containerisierten Plattformen wie Kubernetes wird Software nicht mehr klassisch installiert, sondern als **Container Image** verteilt.
- Container Images bestehen nicht nur aus der eigenen Anwendung, sondern aus vielen **Fremdkomponenten**:
 - Base Images (z. B. nginx, ubuntu, alpine)
 - Betriebssystem-Pakete
 - Bibliotheken und Laufzeitumgebungen
 - Applikationsabhängigkeiten

Jede dieser Komponenten kann **bekannte Sicherheitslücken** enthalten, die sich unbemerkt in die eigene Umgebung einschleusen.

Vulnerability Scanning ist der Prozess, mit dem bekannte Sicherheitslücken in diesen Bestandteilen **systematisch erkannt und bewertet** werden.

Ohne Scanning bleibt unklar, welche Risiken mit einem Image in das Cluster eingebracht werden.

Vulnerability Scanning - Sicherheit von Container Images bewerten

Was genau wird beim Scanning analysiert?

Ein Vulnerability Scanner untersucht nicht den Quellcode, sondern die Inhalte des Images:

- installierte Betriebssystem-Pakete (z. B. apk, apt, rpm)
- Versionen von Bibliotheken und Runtimes
- Applikationsabhängigkeiten (z. B. Python, Node.js, Java)

Diese Informationen werden mit öffentlichen Schwachstellendatenbanken abgeglichen.

Wird eine bekannte Schwachstelle gefunden, wird sie dem Image zugeordnet.

Wichtig:

Scanning bewertet bekannte Schwachstellen – nicht die Qualität des Codes.

Vulnerability Scanning - Sicherheit von Container Images bewerten

Scan Vulnerability - manuell

- Vulnerability-Scan im Kontext der Artefakte verfügbar.

< Projects < docker-hub

library/nginx

Info Artifacts

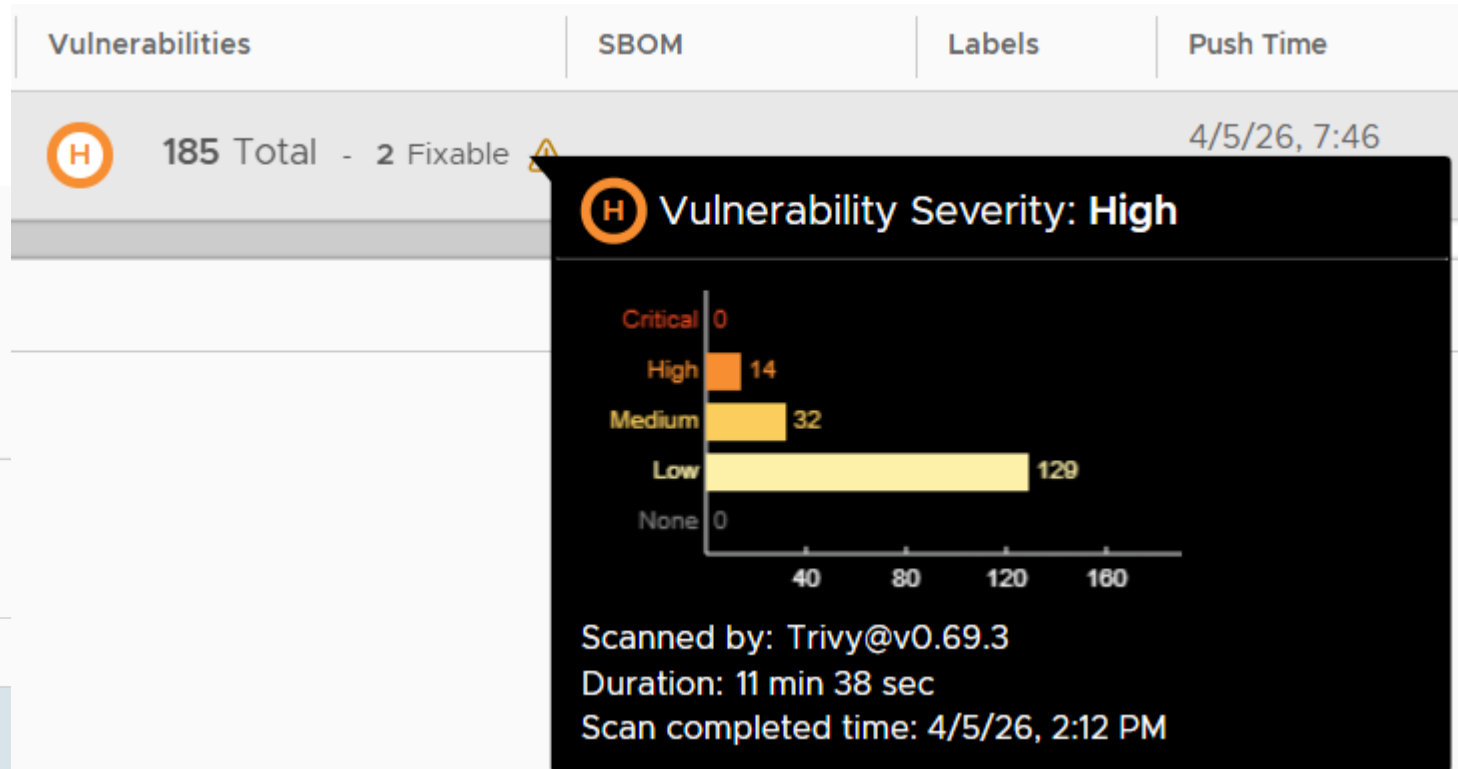
SCAN VULNERABILITY

GENERATE SBOM

ACTIONS ▾

Artifacts Tags Signed

sha256:7150b3a3 1.29



Vulnerability Scanning - Sicherheit von Container Images bewerten

Scan Ergebnisse anschauen

- Artefakt anklicken

[< Projects](#) [< docker-hub](#) [< library/nginx](#)

 sha256:7150b3a3 

Tags

[+ ADD TAG](#) [REMOVE TAG](#) 

☐	Name	Pull Time	Push Time
☐	1.29		4/5/26, 7:46 AM

Page size **15** 1 - 1 of 1 items

Additions

[Vulnerabilities](#) [SBOM](#)

[SCAN VULNERABILITY](#) 

	Vulnerability	Severity	CVSS3	Status	Package	Current version	Fixed in version	Listed In CVE Allowlist
>	CVE-2026-33164	High	nvd: 7.5	affected	libde265-0	1.0.15-1+b3		No

Vulnerability Scanning - Sicherheit von Container Images bewerten

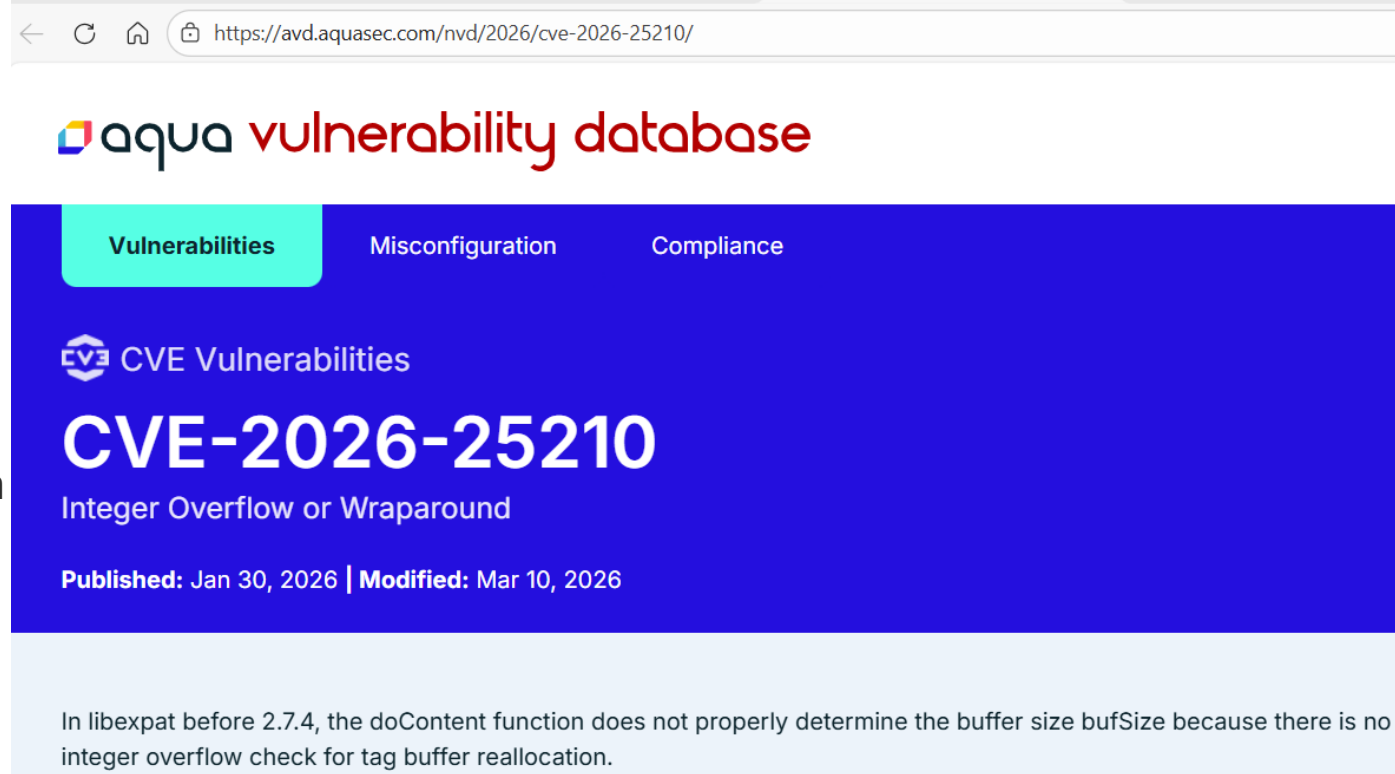
CVEs als Grundlage der Bewertung

Die meisten Scanner arbeiten mit sogenannten CVEs (Common Vulnerabilities and Exposures).
Eine CVE beschreibt eine öffentlich bekannte Sicherheitslücke eindeutig und herstellerübergreifend.

Eine CVE enthält typischerweise:

- eindeutige Kennung (z. B. CVE-2023-0464)
- betroffene Software und Versionen
- Beschreibung der Schwachstelle
- Einschätzung der Schwere

Da CVEs öffentlich sind, sind sie **auch Angreifern bekannt** – nicht nur Sicherheitsteams.



The screenshot shows a web browser window with the URL <https://avd.aquasec.com/nvd/2026/cve-2026-25210/>. The page is titled "aqua vulnerability database" and features a navigation bar with "Vulnerabilities", "Misconfiguration", and "Compliance". The main content area is blue and displays "CVE-2026-25210" in large white text, with the description "Integer Overflow or Wraparound" below it. It also shows the publication date "Published: Jan 30, 2026" and the modification date "Modified: Mar 10, 2026". At the bottom, a light blue box contains the text: "In libexpat before 2.7.4, the doContent function does not properly determine the buffer size bufSize because there is no integer overflow check for tag buffer reallocation."

Vulnerability Scanning - Sicherheit von Container Images bewerten

Schweregrade (Severity) einordnen

Harbor klassifiziert Schwachstellen üblicherweise in mehrere Severity-Stufen:

- Critical – hohe Ausnutzbarkeit und große Auswirkungen
- High – ernstes Risiko, häufig priorisiert zu beheben
- Medium – situationsabhängiges Risiko
- Low – geringe oder theoretische Auswirkung
- Unknown – unvollständig bewertbar

Dabei gilt:

Nicht jede Schwachstelle ist automatisch kritisch – entscheidend ist der Kontext.

Vulnerability Scanning - Sicherheit von Container Images bewerten

Vulnerability Scanning in Harbor

Harbor integriert Scanner (z. B. Trivy) direkt in den Registry-Workflow.

Ein Scan wird typischerweise ausgelöst:

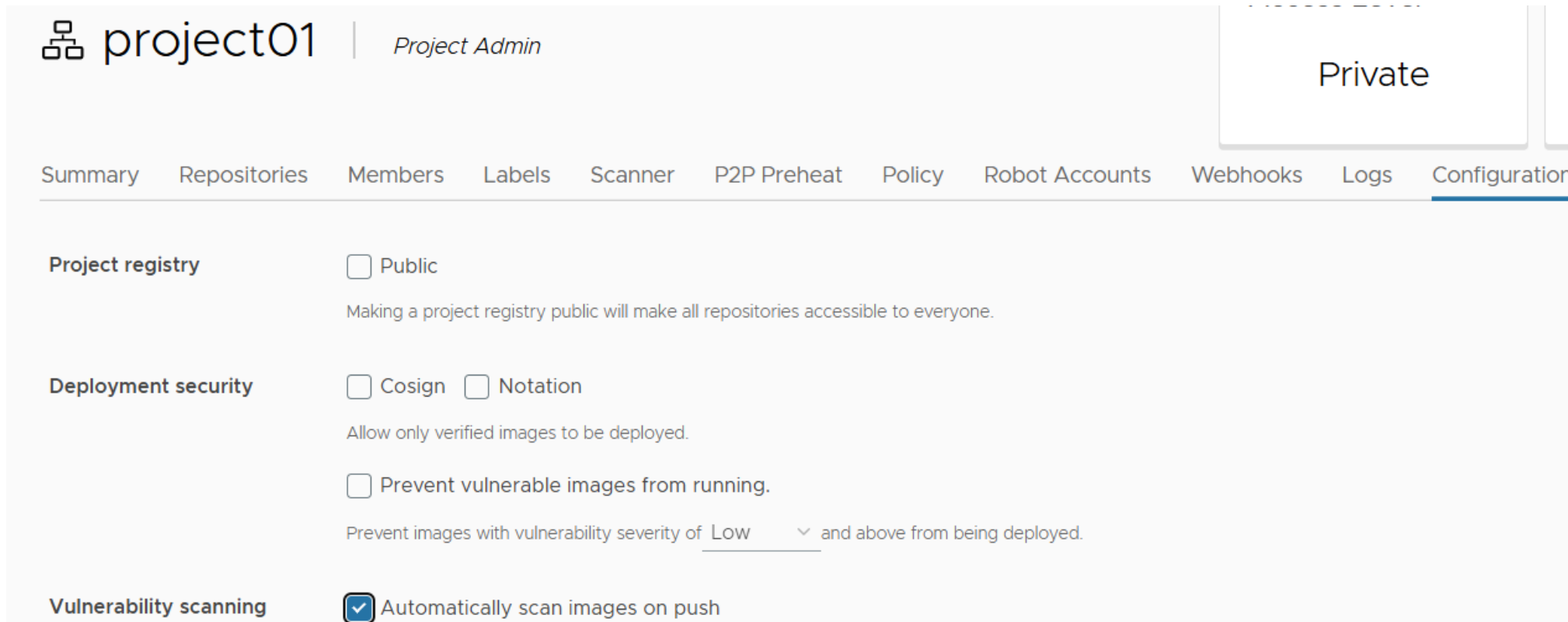
- nach dem Push eines Images
- nach der Replikation aus einer externen Registry
- manuell durch Benutzer

Die Scan-Ergebnisse werden **image- und versionsbezogen** gespeichert und sind jederzeit nachvollziehbar.

Automatisches Vulnerability Scanning

Scan Vulnerability nach einem push

- In Project > Configuration einstellbar.



project01 | Project Admin

Private

Summary Repositories Members Labels Scanner P2P Preheat Policy Robot Accounts Webhooks Logs **Configuration**

Project registry ☐ Public
Making a project registry public will make all repositories accessible to everyone.

Deployment security ☐ Cosign ☐ Notation
Allow only verified images to be deployed.

☐ Prevent vulnerable images from running.
Prevent images with vulnerability severity of Low and above from being deployed.

Vulnerability scanning ☒ Automatically scan images on push

Automatisches Vulnerability Scanning

Nach einem push

- In Logs taucht der Scanner auf.

Logs					
Audit Logs Audit Logs (Legacy)					
Timestamp	Username	Resource	Resource Type	Operation	
4/6/26, 9:16 AM	robot\$project01+scanner- Trivy-8c3bc621-3188-11f1- a2c1-5a7b6ac4ee62	project01/demo@sha256:77c3d3a719b7031063832d24337ac4666bc9cff7f0511baddaf642e128d6a86c	artifact	pull	
4/6/26, 9:16 AM	project01Admin	project01/demo:1.5	artifact	create	

Automatisches Vulnerability Scanning

Nach einem push

- Nach erfolgreichem Scan auch das Ergebnis.

demo

Info
Artifacts

SCAN VULNERABILITY
GENERATE SBOM
ACTIONS

	Artifacts	Tags	Signed	Size	Vulnerabilities	SBOM	Labels	Push Time
☐	 sha256:77c3d3a7 	1.5		24.80MiB	 4 Total - 4 Fixable	No SBOM		4/6/26,
☐	 sha256:8616a489 	1.4		24.80MiB	Not Scanned			
☐	 sha256:1d971710 	1.0		24.80MiB	Not Scanned			
☐	 sha256:f4d41303 			24.80MiB	Not Scanned			

Manage Columns

 Vulnerability Severity: High

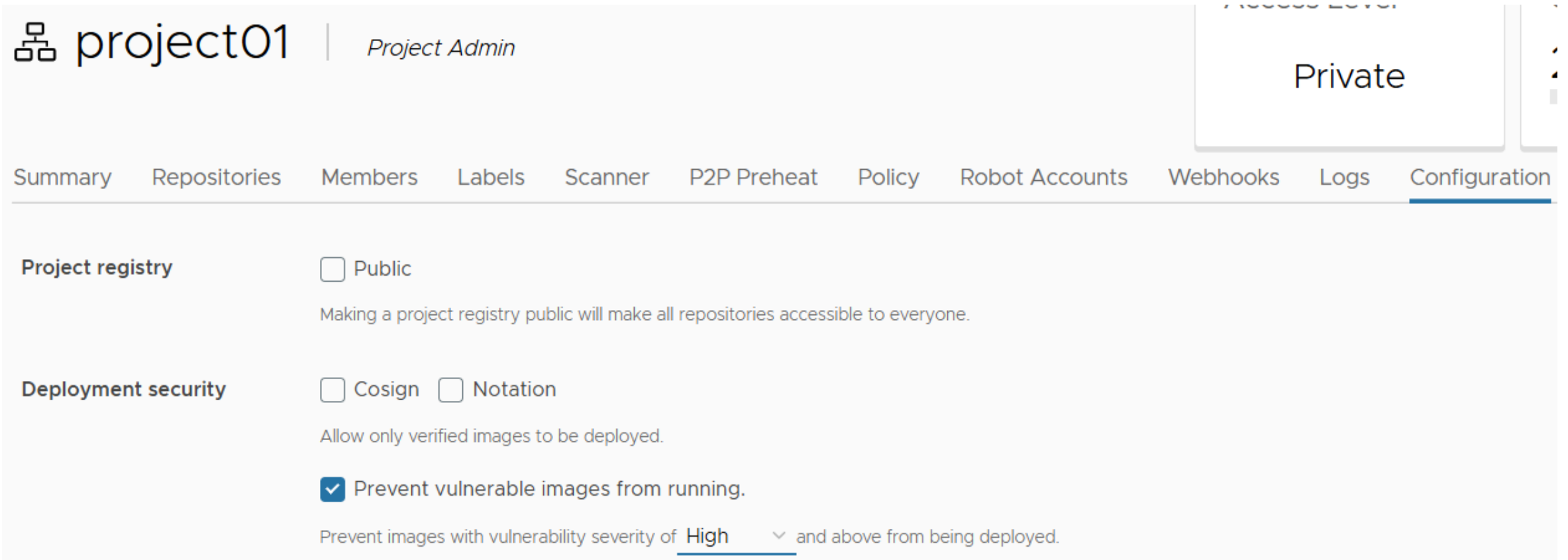
Critical 0
High 3
Medium 1
Low 0
None 0

Scanned by: Trivy@v0.69.3
Duration: 1 min 41 sec
Scan completed time: 4/6/26, 9:18 AM

Automatisches Vulnerability Scanning

Policy konfigurieren

- Ohne Policy kann unsicheres Image gepullt werden!
- Policy aktivieren.



The screenshot shows the Harbor Project Admin interface for 'project01'. The 'Configuration' tab is selected in the top navigation bar. Under 'Project registry', the 'Public' checkbox is unchecked. Under 'Deployment security', the 'Prevent vulnerable images from running' checkbox is checked, and the vulnerability severity is set to 'High'.

project01 | Project Admin

Private

Summary Repositories Members Labels Scanner P2P Preheat Policy Robot Accounts Webhooks Logs Configuration

Project registry ☐ Public

Making a project registry public will make all repositories accessible to everyone.

Deployment security ☐ Cosign ☐ Notation

Allow only verified images to be deployed.

☒ Prevent vulnerable images from running.

Prevent images with vulnerability severity of High and above from being deployed.

Vulnerability Scanning - Sicherheit von Container Images bewerten

Interpretation von Scan-Ergebnissen

Ein Scan liefert unter anderem:

- Anzahl der Schwachstellen pro Severity
- betroffene Pakete und Versionen
- Hinweise auf verfügbare Fixes

Ein typisches Ergebnis zeigt, woher das Risiko stammt – häufig aus dem Base Image oder Systembibliotheken.

Viele Sicherheitsprobleme entstehen nicht durch eigene Entwicklung, sondern durch veraltete Abhängigkeiten.

Vulnerability Scanning - Sicherheit von Container Images bewerten

Policy konfigurieren

- Risiko-Übernahme konfigurieren.

CVE allowlist

Project allowlist allows vulnerabilities in this list to be ignored in this project when pushing and pulling images.

You can either use the default allowlist configured at the system level or click on 'Project allowlist' to create a new allowlist

Add individual CVE IDs before clicking 'COPY FROM SYSTEM' to add system allowlist as well.

☐ System allowlist ☒ Project allowlist

ADD **COPY FROM SYSTEM**

Enter CVE ID(s)

CVE-2026-33416
CVE-2026-33636
CVE-2026-22184
CVE-2026-27171

Separator: commas or newline characters

ADD

Expires at **Never expires** 

☒ Never expires

Vulnerability Scanning - Sicherheit von Container Images bewerten

Grenzen des Vulnerability Scannings

Wichtig für eine realistische Einordnung:

- Keine Erkennung von Zero-Day-Lücken
- Keine Aussage über tatsächliche Ausnutzbarkeit im konkreten Einsatz
- Keine Laufzeit- oder Verhaltensanalyse
- Kein Ersatz für Architektur- oder Code-Reviews

Umgang mit gefundenen Schwachstellen

Typische Maßnahmen sind:

- Aktualisierung des Base Images
- Verwendung schlanker Images
- Aktualisierung von Abhängigkeiten
- Dokumentierte Risikoakzeptanz

Dabei ist Security immer eine **bewusste Entscheidung** zwischen Risiko, Aufwand und Nutzen.

Vulnerability Scanning - Sicherheit von Container Images bewerten

Scans als Grundlage für Governance und Policies

Vulnerability Scanning liefert die Datenbasis für automatisierte Entscheidungen, z. B.:

- Blockierung von Images mit CRITICAL CVEs
- Einschränkung bestimmter Image-Typen
- Freigabeprozesse vor Produktivdeployments

Die eigentliche Durchsetzung erfolgt später über **Policy Enforcement** und **Admission Controller**.

Typische Missverständnisse

- „Keine CVEs = sicher“ → falsch
- „Viele CVEs = unbrauchbar“ → falsch
- „Scanner entscheiden automatisch richtig“ → falsch

Sicherheit erfordert Bewertung – nicht nur Messung.

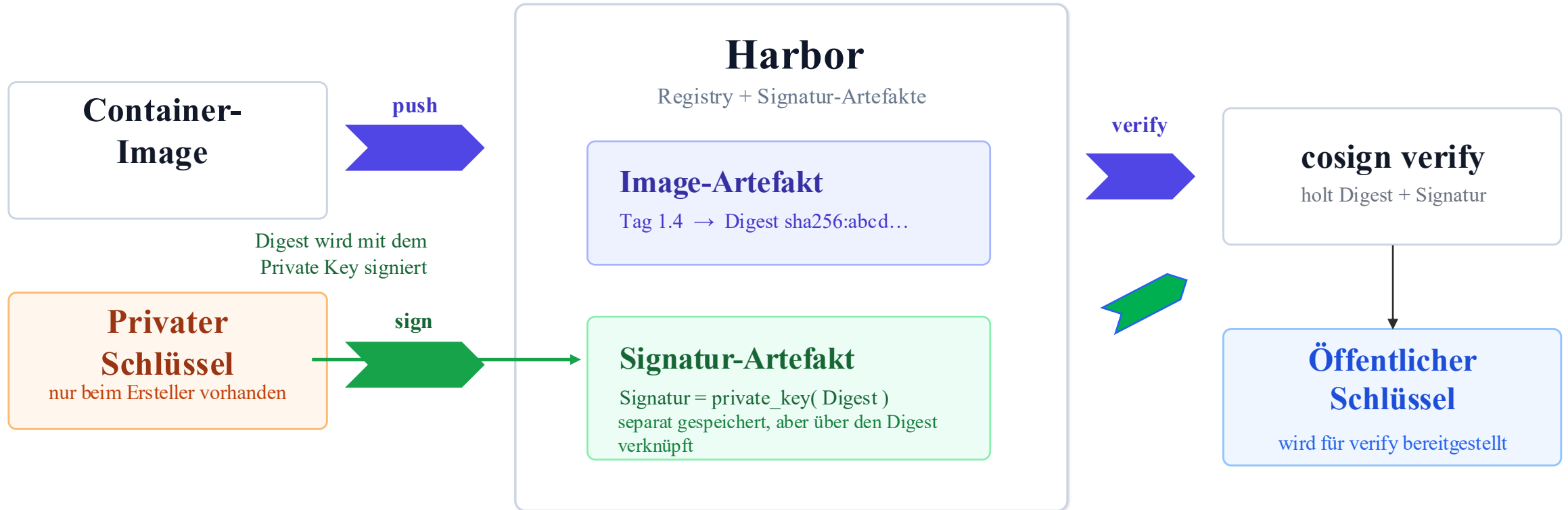
Agenda

Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnerability-Scanner
- Image Signing
- Policy Enforcement

Image Signing – Überblick

Wie Herkunft und Integrität eines Container-Images nachgewiesen werden



Integrität

Bei verify wird geprüft, ob die Signatur genau zu diesem Digest passt. Wurde das Image nach dem Signieren verändert, ändert sich der Digest – die Prüfung schlägt fehl.

Herkunft

Nur der Besitzer des Private Keys kann die gültige Signatur erzeugen. Mit dem Public Key wird nachgewiesen, von wem das Image stammt.

Image Signing – Vertrauenswürdige Container Images in Kubernetes

Einordnung und Motivation

In Kubernetes-basierten Plattformen werden Anwendungen nicht mehr direkt installiert, sondern als Container Images bereitgestellt und automatisiert ausgerollt. Kubernetes selbst prüft dabei standardmäßig lediglich, ob ein Image technisch abrufbar ist – nicht jedoch, ob es aus einer vertrauenswürdigen Quelle stammt oder ob es seit der Erstellung verändert wurde.

Gerade in Umgebungen mit mehreren Teams, automatisierten CI/CD-Pipelines und externen Abhängigkeiten entsteht dadurch ein erhebliches Risiko: Manipulierte oder unerwünschte Images können unbemerkt in ein Cluster gelangen und dort beliebig oft repliziert werden. Klassische Sicherheitsmechanismen greifen hier zu spät oder gar nicht.

Image Signing adressiert genau dieses Problem, indem es die **Herkunft und Integrität von Container Images kryptografisch absichert**. Es ist ein zentraler Baustein moderner Software-Supply-Chain-Security.

Image Signing – Vertrauenswürdige Container Images in Kubernetes

Grundprinzip von Image Signing

Beim Image Signing wird nicht das Container Image selbst verändert. Stattdessen wird der **Digest** des Images – also dessen kryptografischer Fingerabdruck – mit einem privaten Schlüssel signiert. Diese Signatur wird separat gespeichert und kann später jederzeit überprüft werden.

Der entscheidende Punkt ist:

Bereits die kleinste Änderung am Image führt zu einem neuen Digest. Damit wird eine bestehende Signatur automatisch ungültig. Image Signing stellt somit sicher, dass ein Image **exakt dem Zustand entspricht**, den der Ersteller signiert hat.

Im Ergebnis beantwortet Image Signing drei zentrale Fragen:

- Wer hat dieses Image erstellt?
- Wurde das Image seitdem verändert?
- Darf diesem Image vertraut werden?

Image Signing – Vertrauenswürdige Container Images in Kubernetes

Rolle von Harbor beim Image Signing

Harbor fungiert beim Image Signing als **zentraler Vertrauensspeicher** innerhalb der Container-Plattform. Es verwaltet nicht nur Images, sondern auch die zugehörigen **Signaturen** und stellt diese Informationen anderen Systemen – insbesondere Kubernetes – zur Verfügung.

In Harbor sind Signaturen:

- eindeutig einem Image-Digest zugeordnet
- im Webinterface sichtbar (wenn Notary v.2, nicht Cosign)
- versions- und revisionssicher gespeichert

Wichtig ist dabei:

Harbor selbst trifft **keine Laufzeitentscheidungen**. Es entscheidet nicht, ob ein Image gestartet werden darf. Stattdessen stellt Harbor die notwendigen Informationen bereit, damit Kubernetes oder vorgelagerte Sicherheitsmechanismen diese Entscheidung treffen können.

Image Signing – Vertrauenswürdige Container Images in Kubernetes

Technischer Ablauf eines signierten Image-Workflows

Ein typischer Image-Signing-Workflow sieht folgendermaßen aus:

- Zunächst wird das Container Image im Rahmen eines Build-Prozesses erstellt, beispielsweise in einer CI-Pipeline. Nach dem erfolgreichen Build wird das Image in die Harbor Registry gepusht. Dabei entsteht ein eindeutiger Digest, der den exakten Inhalt des Images beschreibt.
- Anschließend wird dieser Digest signiert. Dies erfolgt üblicherweise automatisiert in der Pipeline mithilfe eines Signaturwerkzeugs wie Cosign. Die erzeugte Signatur wird ebenfalls in Harbor abgelegt und mit dem entsprechenden Image verknüpft.

Ab diesem Zeitpunkt ist das Image als „signiert“ gekennzeichnet. Diese Information kann nun von Kubernetes abgefragt und überprüft werden, bevor ein Deployment zugelassen wird.

Image Signing – Vertrauenswürdige Container Images in Kubernetes

Image Signing im Zusammenspiel mit Kubernetes

Kubernetes selbst überprüft Signaturen nicht automatisch. Um Image Signing wirksam durchzusetzen, wird ein zusätzlicher Kontrollmechanismus benötigt. Dies geschieht in der Regel über sogenannte **Admission Controller**.

Admission Controller prüfen neue Ressourcen – etwa ein Deployment – noch bevor sie im Cluster angelegt werden. Dabei können sie:

- das verwendete Image identifizieren
- dessen Digest ermitteln
- die zugehörige Signatur bei Harbor oder über Cosign prüfen

Erst wenn die Signatur gültig ist und den definierten Regeln entspricht, wird das Deployment zugelassen. Andernfalls wird es blockiert, noch bevor ein Pod gestartet wird.

Dieses Zusammenspiel trennt klar die Verantwortlichkeiten:

- Harbor liefert Vertrauensinformationen
- Kubernetes erzwingt die Regeln

Image Signing – Vertrauenswürdige Container Images in Kubernetes

Typische Richtlinien in der Praxis

In realen Umgebungen werden Image-Signing-Policies häufig mit weiteren Kriterien kombiniert. Beispiele sind:

- Nur signierte Images dürfen deployt werden
- Nur Images aus bestimmten Harbor-Projekten sind erlaubt
- Nur Signaturen bestimmter Schlüssel oder Organisationen werden akzeptiert

Dadurch entsteht ein kontrollierter, nachvollziehbarer Freigabeprozess für Container Images – ohne manuelle Eingriffe im Tagesgeschäft.

Grenzen und realistische Erwartungen

Image Signing ist kein Allheilmittel. Es garantiert weder, dass ein Image frei von Schwachstellen ist, noch dass der enthaltene Code korrekt oder sicher implementiert wurde. Vielmehr stellt es sicher, dass genau das Image ausgeführt wird, das freigegeben wurde – nicht mehr und nicht weniger.

Der sichere Umgang mit Schlüsseln ist dabei essenziell. Kompromittierte Signaturschlüssel untergraben das gesamte Vertrauensmodell.

Agenda

Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnerability-Scanner
- Image Signing
 - Cosign-Keys erstellen und speichern
 - Image signieren
 - Image verifizieren
- Policy Enforcement

Image signieren - Cosign-Keys erstellen (1)

```
apiVersion: v1
kind: Pod
metadata:
  name: cosign-generate
  namespace: dev-namespace
spec:
  restartPolicy: Never
  containers:
    - name: keygen
      image: alpine:3.19
      command: [sh, -c]
      args:
        - apk add --no-cache curl
        - curl -L -o /usr/local/bin/cosign https://github.com/sigstore/cosign/releases/download/v2.2.3/cosign-linux-amd64
        - chmod +x /usr/local/bin/cosign
        - export COSIGN_PASSWORD=demo
        - cosign generate-key-pair --output-key-prefix /workspace/cosign
        - echo Keys erzeugt -- halte Container offen...
        - sleep 3600
```

Image signieren - Cosign-Keys erstellen (2)

```
$ kubectl exec --it cosign-generate -- sh
/# ls -la /workspace
total 16
drwxrwxrwx . . . . 2 .root . . . . .root . . . . .4096 Apr . . 7 .03:53 .
drwxr-xr-x . . . . 1 .root . . . . .root . . . . .4096 Apr . . 7 .03:53 .
-rw----- . . . . 1 .root . . . . .root . . . . .653 Apr . . 7 .03:53 cosign.key
-rw-r--r-- . . . . 1 .root . . . . .root . . . . .178 Apr . . 7 .03:53 cosign.pub
/# exit
$ kubectl cp cosign-generate:/workspace/cosign.key ./cosign.key
tar: removing leading '/' from member names
$ kubectl cp cosign-generate:/workspace/cosign.pub ./cosign.pub
```

```
tar: removing leading '/' from member names
$ ls -l
total 12
-rw-rw-r-- . 1 .user01 .user01 .738 Apr . . 7 .03:51 cosign-generate-pod.yaml
-rw-rw-r-- . 1 .user01 .user01 .653 Apr . . 7 .03:55 cosign.key
-rw-rw-r-- . 1 .user01 .user01 .178 Apr . . 7 .03:55 cosign.pub
```

```
$ kubectl create secret generic cosign-key --from-file=cosign.key \
--from-file=cosign.pub
secret/cosign-key created
```

Image signieren - Signieren (1)

```
apiVersion: v1
kind: Pod
metadata:
  name: cosign-sign
  namespace: dev-namespace
spec:
  restartPolicy: Never
  containers:
  - name: cosign
    image: gcr.io/projectsigstore/cosign:v2.2.3
    args:
    - sign
    - --key
    - /keys/cosign.key
    - --allow-insecure-registry
    - --tlog-upload=false
    -
    harbor.local/project01/demo@sha256:83558f1d9cce36445163ac126010394f0b18fef6
    248f0a25e6167753987669d7
    env:
    - name: COSIGN_PASSWORD
      value: demo
    - name: DOCKER_CONFIG
      value: /tmp/.docker
    - name: REGISTRY_AUTH_PLAINHTTP
      value: "true"
```

Image signieren - Signieren (2)

```
.....volumeMounts:[]
.....- .name: .cosign-key[]
.....  .mountPath: ./keys[]
.....  .readOnly: .true[]
[]
.....- .name: .harbor-auth[]
.....  .mountPath: ./tmp/.docker[]
.....  .readOnly: .true[]
[]
. .volumes:[]
. ....- .name: .cosign-key[]
. ....  .secret:[]
. ....  .secretName: .cosign-key[]
[]
. ....- .name: .harbor-auth[]
. ....  .secret:[]
. ....  .secretName: .harbor-registry[]
. ....  .items:[]
. ....    - .key: .dockerconfigjson[]
. ....    .path: .config.json[]
```

Image signieren - Signieren (3)

```
$ ·kubectl·logs·cosign-sign℥  
Pushing·signature·to:·harbor.local/project01/demo℥
```

Timestamp	Username	Resource	Resource Type	Operation
4/7/26, 8:18 AM	project01Admin	project01/demo:sha256-83558f1d9cce36445163ac126010394f0b18fef6248f0a25e6167753987669d7.sig	artifact	pull
4/7/26, 8:11 AM	project01Admin	project01/demo:sha256-83558f1d9cce36445163ac126010394f0b18fef6248f0a25e6167753987669d7.sig	artifact	create

Image signieren - Verifizieren (1)

```
apiVersion: v1
kind: Pod
metadata:
  name: cosign-verify
  namespace: dev-namespace
spec:
  restartPolicy: Never
  containers:
    - name: cosign
      image: gcr.io/projectsigstore/cosign:v2.2.3
      args:
        - verify
        - --key
        - /keys/cosign.pub
        - --allow-insecure-registry
        - --insecure-ignore-tlog
        - harbor.local/project01/demo@sha256:83558f1d9cce36445163ac126010394f0b18fef6248f0a25e6167753987669d7
```

```
      env:
        - name: DOCKER_CONFIG
          value: /tmp/.docker
        - name: REGISTRY_AUTH_PLAINHTTP
          value: "true"
```

Image signieren - Verifizieren (2)

```
.....volumeMounts:[]
.....- name: cosign-key[]
.....  mountPath: /keys[]
.....  readOnly: true[]
[]
.....- name: harbor-auth[]
.....  mountPath: /tmp/.docker[]
.....  readOnly: true[]
[]
  volumes:[]
  ....- name: cosign-key[]
  ....  secret:[]
  ....  secretName: cosign-key[]
[]
  ....- name: harbor-auth[]
  ....  secret:[]
  ....  secretName: harbor-registry[]
  ....  items:[]
  ....    - key: .dockerconfigjson[]
  ....    path: config.json[]
```

Image signieren - Verifizieren (3)

```
$ kubectl logs cosign-verify
WARNING: Skipping tlog verification is an insecure practice that lacks of
transparency and auditability verification for the signature.
Verification for
harbor.local/project01/demo@sha256:83558f1d9cce36445163ac126010394f0b18fef6
248f0a25e6167753987669d7 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - The signatures were verified against the specified public key
[{"critical":{"identity":{"docker-
reference":"harbor.local/project01/demo"},"image":{"docker-manifest-
digest":"sha256:83558f1d9cce36445163ac126010394f0b18fef6248f0a25e6167753987
669d7"},"type":"cosign.container.image.signature"},"optional":null}]
```

Image Signing – Ausblick

Umsetzen der Signatur-Sicherheit im Kubernetes Cluster mit ‚Kyverno‘

Was ist Kyverno?

Kyverno ist eine **Kubernetes-native Policy Engine**, mit der sich **Sicherheits- und Governance-Regeln direkt in Kubernetes durchsetzen lassen**.

Der Name bedeutet:

Kyverno = „govern“ (altgriechisch)

Kerngedanke:

Regeln als Kubernetes-Ressourcen, nicht als externe Logik.

Policies werden in **YAML** definiert – genau wie Deployments, Services oder Ingresses.

Kyverno wird selbst, meist per Helm, im Cluster installiert und arbeitet als **Admission Controller**.

```
> helm repo add kyverno https://kyverno.github.io/kyverno/  
helm install kyverno kyverno/kyverno \  
  --namespace kyverno \  
  --create-namespace
```

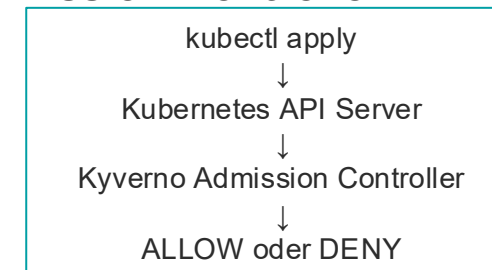


Image Signing – Ausblick

Beispiel: Nur signierte Images erlauben

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-signed-images
spec:
  validationFailureAction: Enforce
  rules:
    - name: verify-image-signature
      match:
        resources:
          kinds:
            - Pod
      verifyImages:
        - imageReferences:
            - "harbor.example.com/demo/*"
          key: |-
            -----BEGIN PUBLIC KEY-----
            ...
            -----END PUBLIC KEY-----
```

Unsigned Image = Pod wird blockiert

Angriff	Wirkung
Image-Manipulation	✗ blockiert
Fake Images	✗ blockiert
Unautorisierte Builds	✗ blockiert
Registry-Missbrauch	✗ blockiert

Supply-Chain-Security wird erzwingbar.

Agenda

Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnerability-Scanner
- Image Signing
- Policy Enforcement

Policy Enforcement - Sicherheitsregeln konsequent durchsetzen

Einordnung: Was bedeutet Policy Enforcement?

Policy Enforcement beschreibt das **automatische Durchsetzen von Regeln**, die festlegen:

- was erlaubt ist
- was verboten ist
- unter welchen Bedingungen Workloads in Kubernetes laufen dürfen

Wichtig:

Eine Policy ist wertlos, wenn sie nicht erzwungen wird.

Policy Enforcement - Sicherheitsregeln konsequent durchsetzen

Warum Policy Enforcement notwendig ist

In modernen Kubernetes-Umgebungen gilt:

- viele Teams
- häufige Deployments
- automatisierte CI/CD-Pipelines

Manuelle Kontrollen sind dabei:

- zu langsam
- fehleranfällig
- nicht skalierbar

➔ Policies ersetzen Vertrauen durch überprüfbare Regeln.

Policy Enforcement - Sicherheitsregeln konsequent durchsetzen

Typische Risiken ohne Policy Enforcement

Ohne durchgesetzte Policies kann:

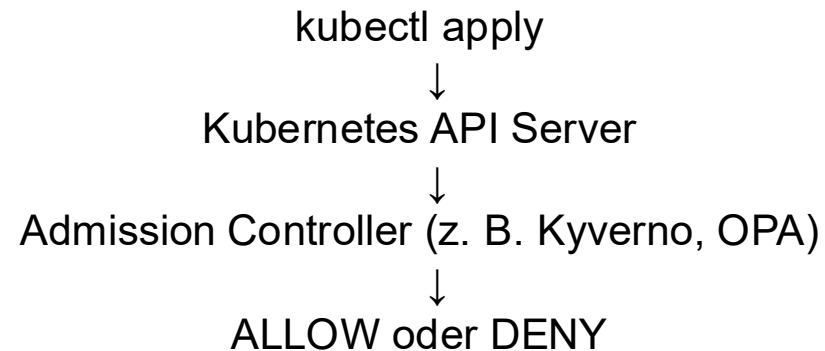
- ein unsicheres Image deployt werden
- ein nicht signiertes Image in Produktion gelangen
- ein Image aus einer fremden Registry gestartet werden
- ein veraltetes, verwundbares Image laufen

➔ Harbor kann diese Risiken sichtbar machen – Kubernetes muss sie verhindern.

Policy Enforcement - Sicherheitsregeln konsequent durchsetzen

Wo Policy Enforcement technisch stattfindet

Policy Enforcement erfolgt in Kubernetes über den **Admission-Prozess**.



➔ Die Entscheidung fällt vor dem Start des Pods.

ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Zentrale Paderborn
Karl-Schurz-Straße 19a
33100 Paderborn
Tel.: 05251 1063-0
Fax: 0180 1 67349 0

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00

seminare@ordix.de
<https://seminare.ordix.de/>

Vielen Dank für Ihre Aufmerksamkeit !